性能测试

比较与实现

在同一 KunminghuV2Config 上，比较自定义 vdot 与标准 RVV 指令组合完成 16 对 signed int8 点积的周期数。两者输入、依赖链长度和输出布局相同，输出低 32 位为和、高 96 位为零。测试同时开启 DiffTest，由独立 C 标量参考检查完整结果。

基础数据位于 tests/bench/kernels.S：vwmul.vv 把 16 对字节扩宽相乘，16 个 int16 乘积存入 v16/v17；切换到 e16、m2 后，用 vwredsum.vs 将结果扩宽归约为 int32；再切回 e8、m1，并将结果移回 v8，作为下一轮输入。v20 的初值和 v24 的尾部在计时前清零，tu 策略使结果高位保持为零。

```
vwmul.vv v16,v8,v9
vsetivli zero,16,e16,m2,tu,mu
vwredsum.vs v24,v16,v20
vsetivli zero,16,e8,m1,tu,mu
vmv.v.v v8,v24
```

自定义路径执行 vdot.vv v10,v8,v9，再用 vmv.v.v v8,v10 建立相同的下一轮依赖。计时前分别运行五组全零和极值输入，包括 (-128)×(-128) 的 +262144 与 (-128)×127 的 -260096，二者路径均通过完整 128 位比较。

tests/bench/main.c 测量 1、8、32 次点积依赖链。每种长度先随机，再采样五次，交替执行基线或 vdot。每次结果均成为下一次输入；每个样本都检查完整结果，全部测量结束后再统一打印。15 组原始数据保存在 性能测试.log，解析后的原始行和中位数保存在 性能数据.json。

周期由被测 RISC-V 程序的 rdcycle 读取。

窗口	计入内容
计算窗口	两次内部 rdcycle 之间的依赖链、循环、必要的 vtype 切换及边界 fence 开销
完整调用窗口	从向量配置和装载前，到结果存储后；额外计入装载、初值设置、结果存储及对应边界开销
样本汇总	每种长度各取五次样本的中位数；加速比=基线周期/vdot 周期

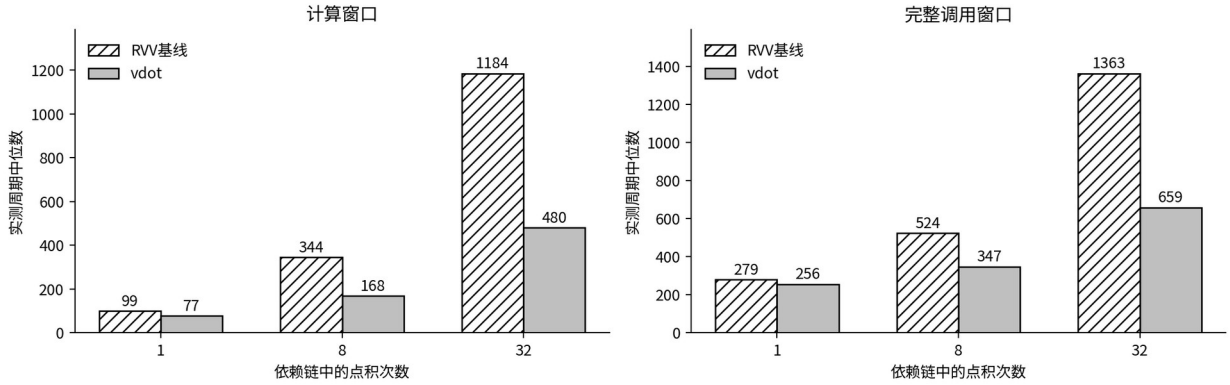
实际周期结果

点积次数	基线计算	vdot 计算	计算比值	基线完整	vdot 完整	完整比值
1	99	77	1.286×	279	256	1.090×
8	344	168	2.048×	524	347	1.510×
32	1184	480	2.467×	1363	659	2.068×

表中周期均为五次采样中位数。1 次点积的计算从 99 周期降至 77 周期，完整实现从 279 降至 256。32 次时，计算窗口从 1184 降至 480，完整窗口从 1363 降至 659。长链中的配置切换和标准归约开销反复出现，vdot 路径的周期差随点积次数增大。

8 次点积的 vdot 计算样本范围为 168～180 周期，其中四次为 168 或 169，另一次为 180；五次中位数为 168。32 次点积的基础范围为 1184～1185，vdot 为 480～481。

同一昆明湖V2配置：每种长度暖机后采样五次



由波形观察指令执行过程

波形程序 tests/smoke/main.c 与性能测试使用只执行一次基础测试和一次 vdot。输入低八对为零、上八对非零，两种结果均为 -32416，高 96 位为零。

目标	输入 sim_timer	输出 sim_timer	观察
vwmul.vv	4636、4637	4638、4639	两个微操作
vwredsum.vs	4643、4644 4651、4658	4645、4646 4653、4660	四个微操作，最后一次输出最终结果
vdot.vv	5020	5022	ROB=(0,105)，完整结果一次输出
vdot 提交	—	5024	由原有 ROB 提交

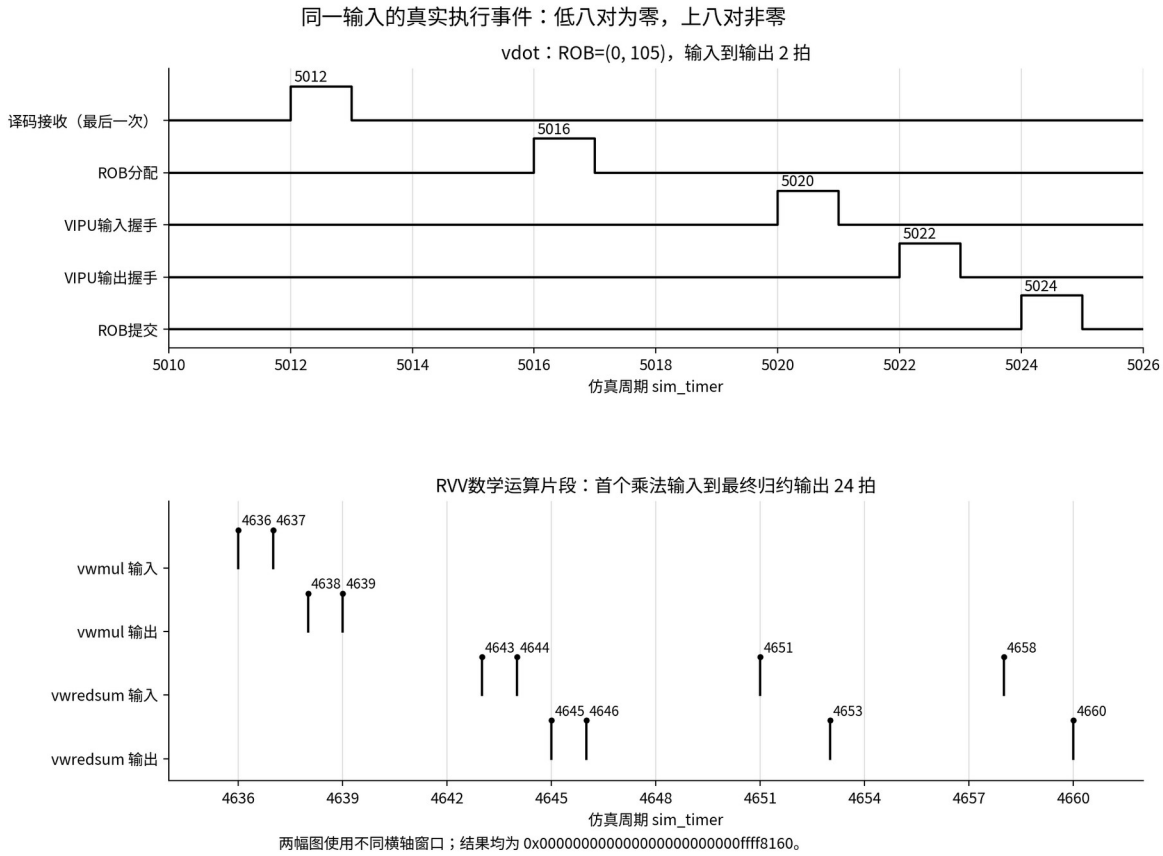


图 1 实际波形提取的握手及提交事件，两幅子图使用不同横轴窗口

vdot 的输入到输出间隔为两拍。基线数学片段从首个乘法输入 4636 到最终归约输出 4660，跨度为 24 拍，包含多个微操作和依赖等待。该跨度未包含随后恢复 vtype 与移动结果的全部开销；程序计时窗口还包含循环与 fence。波形的 24 拍与两拍用于说明具体执行片段，程序加速比采用前表的独立采样结果。

GTKWave 横轴使用 VCD 刻度，本次 sim_timer 比 VCD 数值大 3，两个坐标的差量一致。图中使用 sim_timer 表示仿真周期；VCD 文件的 ps 刻度没有经过芯片时钟频率标定。

复现与查看波形

```
# 在克隆后的作品仓库根目录执行
source scripts/env.sh
bash scripts/build_emu.sh
bash scripts/build_nemu.sh
bash scripts/build_app.sh bench
bash scripts/run_emu.sh bench custom diff
bash scripts/build_app.sh smoke
bash scripts/run_emu.sh smoke custom diff \
  --dump-wave --wave-path "$KKS_ROOT/build/stage3-full.vcd" -b 3000 -e 9000
python3 scripts/analyze_stage3.py wave build/stage3-full.vcd
python3 scripts/analyze_stage3.py perf build/logs/bench-custom-diff.log
python3 scripts/analyze_stage3.py figures
```

完整流程也可通过 scripts/stage3-run.sh 执行。分析脚本检查样本数量、程序输出、GOOD TRAP 和真实退出码，再生成图形。保留的精简波形包含 152 个实际信号；其数值变化来自同次完整 VCD。

```
gtkwave "$KKS_ROOT/stages/03-verification/waves/执行波形.vcd.gz" \
  --script="$KKS_ROOT/stages/03-verification/waves/view.tcl"

KKS_WAVE_VIEW=baseline gtkwave "$KKS_ROOT/stages/03-verification/waves/执行波形.vcd.gz" \
  --script="$KKS_ROOT/stages/03-verification/waves/view.tcl"
```

测量范围

本次测量对象为有真实数据依赖的点积序列。结果包含该处理器的调度、向量配置、寄存器移动、循环与计时开销，不能按两拍单元延迟直接推算整段程序周期。没有进行独立指令峰值吞吐、综合时序、面积或功耗测量。

```
BASELINE_EQ PASS cases=5 full128=checked signed_extremes=checked
BENCH PASS signed_dot16=equivalent samples=15 results128=checked
HIT GOOD TRAP at pc = 0x80000666
KKS_RUN_EXIT code=0
```